

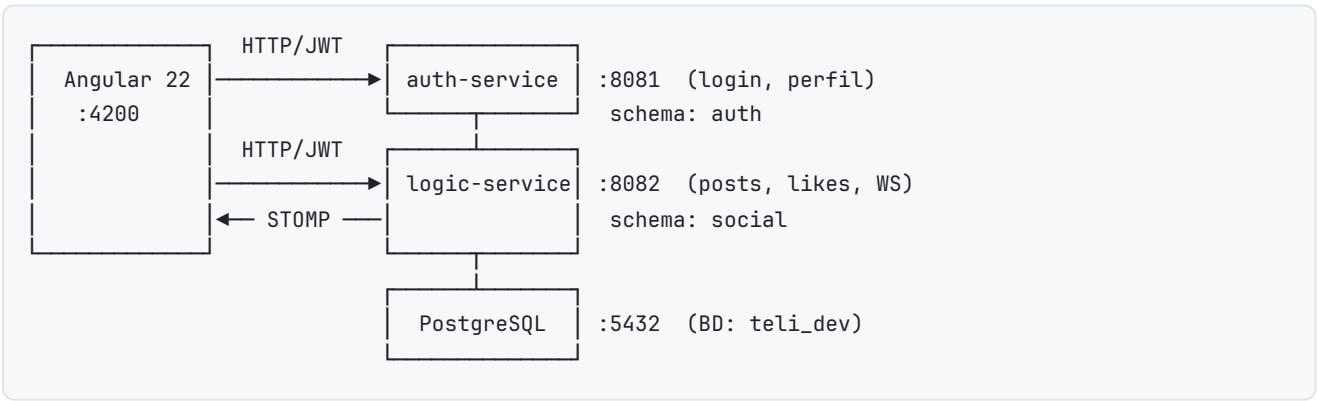
Teli · Red Social (Prueba Técnica Full Stack)

Red social **Teli** con arquitectura de **microservicios**: autenticación con JWT, perfiles, publicaciones y **likes en tiempo real** (WebSocket/STOMP). Todo desplegable con `docker compose up`.

Stack

Capa	Tecnología
Frontend	Angular 22 (standalone, SSR scaffold), NgRx SignalStore , Tailwind v4, <code>@stomp/stompjs</code> + SockJS
Backend	Spring Boot 3.4 · Java 21 · Gradle , arquitectura hexagonal (ports & adapters)
Auth	JWT HS256 (jjwt), BCrypt
Tiempo real	WebSocket + STOMP (<code>/topic/posts.likes</code>)
Base de datos	PostgreSQL 16 · JPA/Hibernate + 2 <code>PROCEDURE</code> PL/pgSQL
Docs	Swagger UI en <code>/docs</code> (OpenAPI 3)

Arquitectura



Los dos servicios comparten una `JWT_SECRET`: `auth-service` firma el token y `logic-service` lo valida. Un solo Postgres con dos schemas (`auth` , `social`).

Estructura del repositorio

<code>periferia-it-challenge/</code>	
<code>├─ docker-compose.yml</code>	Orquesta BD + 3 servicios
<code>├─ database/init.sql</code>	Schemas, tablas, procedures y seed
<code>├─ periferiait-auth-service/</code>	Microservicio de autenticación (hexagonal)
<code>├─ periferiait-logic-service/</code>	Microservicio de publicaciones/likes (hexagonal)

periferiait-teli/	Frontend Angular 22
postman/	Colecciones + environment
tools/ws-test.html	Utilidad para probar el WebSocket

Cada microservicio sigue arquitectura hexagonal:

domain/	modelo puro + excepciones (sin frameworks)
application/	port/in (casos de uso), port/out (puertos), service (implementaciones)
infrastructure/	adapter/in/web (REST), adapter/out (JPA, seguridad, mensajería), config

Requisitos previos

- **Docker + Docker Compose** (opción recomendada), o para desarrollo manual:
- JDK 21, y Node 22 (para el frontend). PostgreSQL 16 si no usas Docker.

Opción A — Levantar todo con Docker (recomendado)

```
docker compose up --build
```

Servicios expuestos:

- Frontend: <http://localhost:4200>
- Auth API + Swagger: <http://localhost:8081/docs>
- Logic API + Swagger: <http://localhost:8082/docs>
- PostgreSQL: localhost:5432 (BD teli_dev)

El database/init.sql se ejecuta automáticamente **la primera vez** (crea schemas, tablas, procedures y datos seed). Para re-sembrar desde cero:

```
docker compose down -v && docker compose up --build
```

Opción B — Desarrollo manual

1. **Base de datos:** crea la BD y ejecuta el script.

```
psql -U postgres -c "CREATE DATABASE teli_dev;"
psql -U postgres -d teli_dev -f database/init.sql
```

2. **auth-service** (puerto 8081):

```
cd periferiait-auth-service
DB_NAME=teli_dev DB_USER=postgres DB_PASSWORD=post1842 ./gradlew bootRun
```

3. **logic-service** (puerto 8082): igual, en su carpeta.

4. **frontend** (puerto 4200):

```
cd periferiait-teli && npm install && npm start
```

Variables de entorno de los servicios: `DB_HOST` , `DB_PORT` , `DB_NAME` , `DB_USER` , `DB_PASSWORD` , `JWT_SECRET` , `CORS_ALLOWED_ORIGINS` (logic).

Usuarios de prueba

Todos con contraseña **Periferia123*** :

Usuario	Alias	Nombre
jdoe	juanp	Juan Pérez
mgarcia	mariag	María García
crodriguez	carlosr	Carlos Rodríguez
alopez	analog	Ana López
dmartinez	diegom	Diego Martínez

Cada uno tiene una publicación seed.

Endpoints principales

auth-service (:8081)

- `POST /auth/login` — emite JWT · `GET /users/me` — perfil (requiere Bearer)

logic-service (:8082)

- `GET /posts` — lista con total de likes
- `POST /posts` — crea publicación (fecha por defecto en BD)
- `POST /posts/{postId}/likes` — toggle de like (difunde por WebSocket)

Documentación interactiva: `/docs` en cada servicio.

Tiempo real (likes)

Al dar like, el nuevo total se difunde por STOMP en `/topic/posts.likes` (endpoint SockJS `/ws`). El frontend actualiza el contador en todos los clientes conectados sin recargar. Se puede probar sin frontend con `tools/ws-test.html` .

Base de datos y procedures

- ORM: JPA/Hibernate. `ddl-auto: validate` (la BD la crea `init.sql`).
- Procedimientos PL/pgSQL (requisito de mínimo 2):
 - `social.sp_create_post` — inserta una publicación.
 - `social.sp_toggle_like` — da/quita like (+ auditoría en `social.like_audit`).
 - Funciones de apoyo: `social.fn_post_like_count`, `auth.fn_get_profile`.

Postman

Importa desde `postman/` :

- `periferiait-auth-service.postman_collection.json`
- `periferiait-logic-service.postman_collection.json`
- `periferiait-local.postman_environment.json`

Ejecuta **Auth** → **Login** (guarda el JWT en la variable global `authToken`); el resto de requests lo reutilizan automáticamente.

Pruebas

```
cd periferiait-auth-service && ./gradlew test
cd periferiait-logic-service && ./gradlew test
cd periferiait-teli && npm test
```

Tests unitarios de la capa de aplicación con puertos mockeados (Mockito), sin Spring ni BD — posible gracias a la arquitectura hexagonal.

Cumplimiento de requisitos (validación cruzada)

Contraste de cada requisito del enunciado contra lo implementado, con su ubicación en el código para su verificación.

Alcance funcional

Requisito	Estado	Dónde
Autenticación (usuario y clave)	✓	<code>POST /auth/login</code> + BCrypt
Ver publicaciones (de otros usuarios)	✓	<code>GET /posts</code> + pantalla Publicaciones
Crear publicación (mensaje, usuario, fecha)	✓	<code>POST /posts</code> (autor del JWT, fecha default en BD)
Ver perfil (Nombres, Apellidos, F. nacimiento, Alias)	✓	<code>GET /users/me</code> + pantalla Perfil

Requisito	Estado	Dónde
Dar likes y verlos en tiempo real (mqtt ó websocket)	✓	WebSocket/STOMP → <code>/topic/posts.likes</code>

Backend (Spring Boot)

Requisito	Estado	Nota
Autenticación: login con JWT	✓	El enunciado sugiere <code>GET</code> ; se usa <code>POST</code> (buena práctica: credenciales fuera de la URL)
Crear publicación (POST)	✓	<code>POST /posts</code>
Listar publicaciones (GET)	✓	<code>GET /posts</code>
Envío de like (POST)	✓	<code>POST /posts/{postId}/likes</code>
Ver perfil (GET)	✓	<code>GET /users/me</code>
Seeder: usuarios de prueba + 1 publicación c/u	✓	<code>database/init.sql</code> (automático en Docker)
Contenedores: Dockerizar los microservicios	✓	<code>Dockerfile</code> en cada servicio

Base de datos

Requisito	Estado	Dónde
PostgreSQL + ORM	✓	JPA / Hibernate
Mínimo 2 <code>PROCEDURE</code> PL/pgSQL	✓	<code>sp_create_post</code> , <code>sp_toggle_like</code> (+ funciones <code>fn_post_like_count</code> , <code>fn_get_profile</code>)

Frontend

Requisito	Estado	Dónde
Pantalla de Login	✓	<code>features/login</code>
Pantalla de Perfil	✓	<code>features/profile</code>
Pantalla de Publicaciones (lista + like + total)	✓	<code>features/posts</code>
Crear publicación (mensaje + fecha default)	✓	<code>features/posts</code> (fecha en BD)
Manejo de estado: NgRx SignalStore (Singleton + Signals)	✓	<code>AuthStore</code> , <code>PostsStore</code> (<code>providedIn: 'root'</code>)

Extras valorados

Extra	Estado	Dónde
Uso de TypeScript y Java	✓	Frontend TS · Backend Java 21
Documentación en Swagger (/docs por servicio)	✓	springdoc-openapi
Pruebas unitarias de código	✓	Backend: JUnit + Mockito (capa de aplicación). Frontend: spec base
Manejo de errores con buenas prácticas	✓	@RestControllerAdvice en ambos servicios
Logs y auditoría en los microservicios	✓	SLF4J + tabla social.like_audit
Observabilidad	✓	Spring Boot Actuator (/health , /info , /metrics)

Entregables

Entregable	Estado	Dónde
Repositorio con backend y frontend	✓	Este repositorio (monorepo)
Docker Compose	✓	docker-compose.yml
Script BD con usuarios predefinidos	✓	database/init.sql
Documentación Swagger	✓	/docs en cada servicio
PDF de instalación y explicación	✓	Este README.md (exportable a PDF)
Video demostrativo		Adjunto a la entrega

Notas de diseño

- **Spring Boot 3.4 / Java 21**
- **Swagger con springdoc-openapi**: el spec se genera automáticamente desde los controllers (/v3/api-docs) y la UI se expone en /docs — sin YAML a mano ni controllers extra. El botón *Authorize* usa el esquema JWT (bearerAuth).
- **JWT compartido por secret** (HS256) entre servicios, sin llamadas entre ellos: logic-service denormaliza authorAlias desde los claims.
- **Procedures solo con parámetros IN** : pgjdbc no soporta extraer parámetros de salida UUID en un CALL ; el id se genera en la app y el total de likes se lee con fn_post_like_count .
- **Frontend en RenderMode.Client** : evita ejecutar código de navegador (localStorage/SockJS) durante el prerender de SSR (detalle abajo).

Frontend: RenderMode.Client y seguridad

El scaffold traía SSR con RenderMode.Prender para todas las rutas. En ese modo Angular ejecuta la app dentro de Node durante el build para generar HTML estático; pero la app toca APIs

que solo existen en el navegador (`localStorage` en el `AuthStore` , `SocketJS` / `window` en el `RealtimeService`), que en Node no existen → el prerender falla. Además, prerenderizar no aporta nada a una app **detrás de login** (no hay contenido público que cachear). Por eso todas las rutas pasan a `RenderMode.Client` : el servidor Node ya no renderiza la app, solo sirve el *shell* y los estáticos, y todo se renderiza en el navegador (SPA).

Implicaciones de seguridad — lo que mejora:

- Sin "state bleed" entre usuarios. En modo `Server` , los servicios `providedIn: 'root'` son singletons compartidos en el proceso Node entre todas las requests; guardar estado por-usuario ahí (token, perfil) puede filtrar datos de un usuario en la respuesta de otro. En `Client` no hay render por-usuario en el servidor → esa clase de bug desaparece.
- El JWT nunca pasa por el servidor de render (vive solo en el navegador): no queda en logs, ni en cachés de páginas SSR, ni en HTML compartido.
- Menor superficie de ataque del servidor: el Node queda como servidor de estáticos, no ejecuta el código de la app por request.

Lo que NO cambia (honestidad):

- `RenderMode.Client` no endurece el almacenamiento del token. El JWT sigue en `localStorage` , accesible por cualquier JS de la página → **vulnerable a XSS**, igual en cualquier modo. Mitigación real (fuera del alcance): cookie `HttpOnly` + `Secure` + `SameSite` en vez de `localStorage` (implica manejar CSRF), y una **CSP** estricta.
- **CSRF**: al usar Bearer token en header (no cookies de sesión) no hay CSRF clásico; el control de origen lo da **CORS**, ya configurado en ambos servicios.